

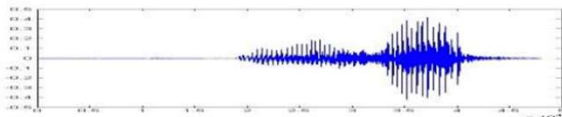
视听信息系统导论

从样本中学习



- 机器学习 $\approx$ 寻找一个函数

- 语音识别

$$f(\text{ ) = \text{"How are you"}$$

- 图片识别

$$f(\text{ ) = \text{"Cat"}$$

- 下围棋

$$f(\text{ ) = \text{"5-5"}$$

- 对话系统

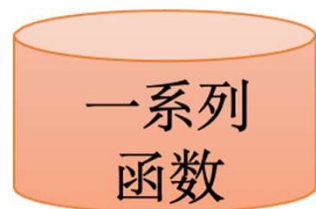
$$f(\text{ "Hi" }) = \text{"Hello"}$$

- 机器学习的框架（以图像分类作为例子）

- 任务：找到一个适当的函数使得

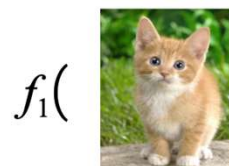
$$f(\text{img_cat}) = \text{"cat"}$$

- 模型：一系列函数



模型

$f_1, f_2 \dots$



$$f_1(\text{img_cat}) = \text{"cat"}$$



$$f_2(\text{img_cat}) = \text{"money"}$$



$$f_1(\text{img_dog}) = \text{"dog"}$$



$$f_2(\text{img_dog}) = \text{"snake"}$$

- 机器学习的框架（以图像识别作为**例子**）

- 数据

- 每个数据集由一个个样本（example, sample）组成，大多时候，它们遵循独立同分布(independently and identically distributed, i.i.d.)。样本有时也叫做数据点（data point）或者数据实例（data instance），通常每个样本由一组称为特征（features, 或协变量（covariates））的属性组成。机器学习模型会根据这些属性进行预测。在上面的监督学习问题中，要预测的是一个特殊的属性，它被称为标签（label），或目标（target）



“monkey”



“cat”

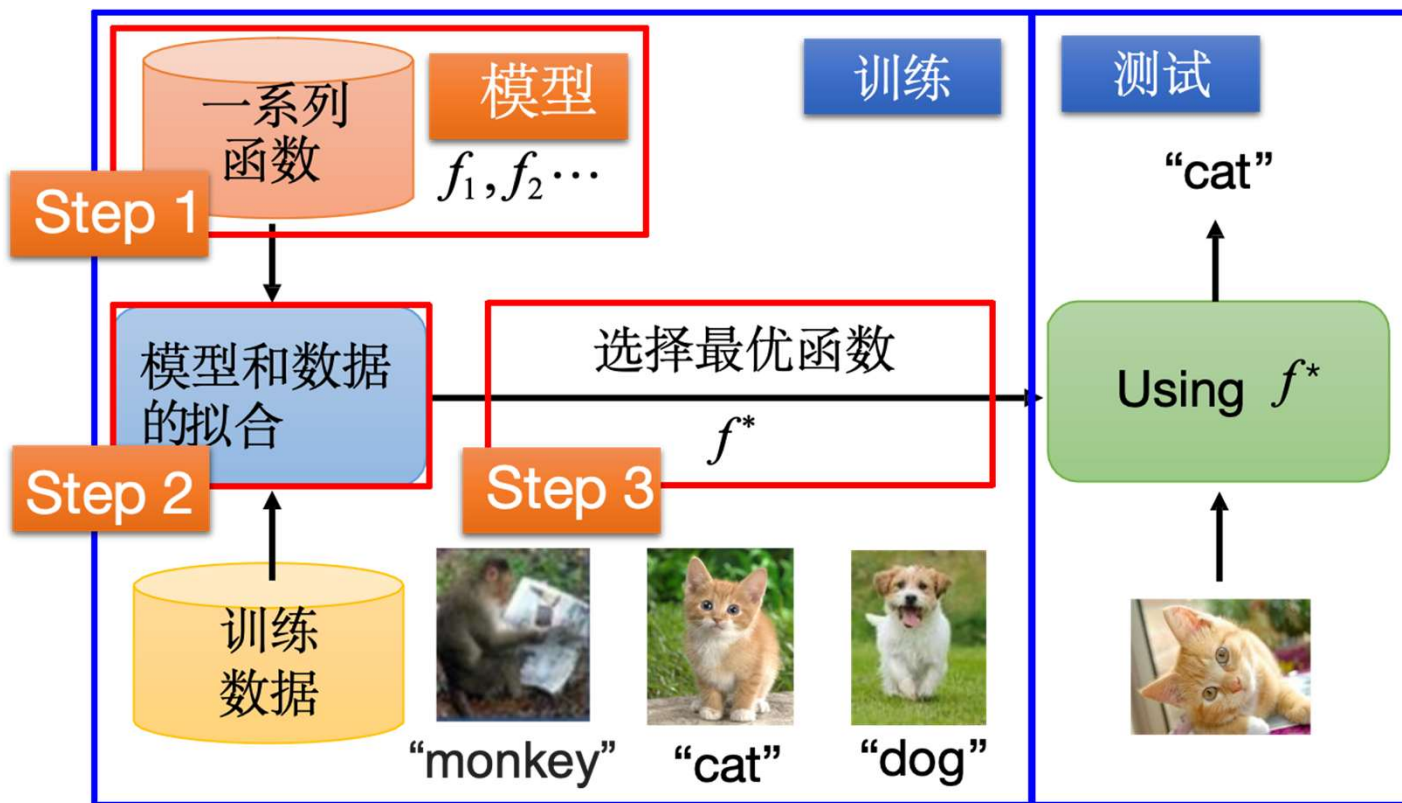


“dog”

- 机器学习的框架（以图像识别作为例子）
 - 训练：从众多函数中挑选出表现最好的



- 机器学习的框架（以图像识别作为例子）
 - 测试：利用上述挑出的函数对没有见过的图片做预测

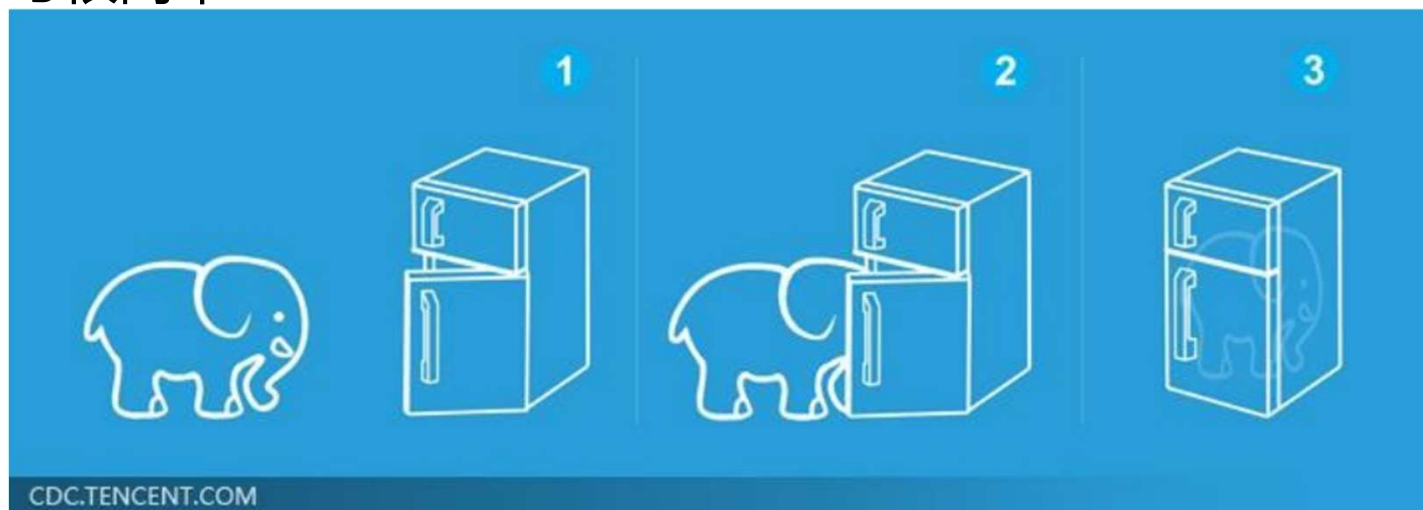


- 机器学习的框架（以图像识别作为例子）

- 总结



- 机器学习很简单！

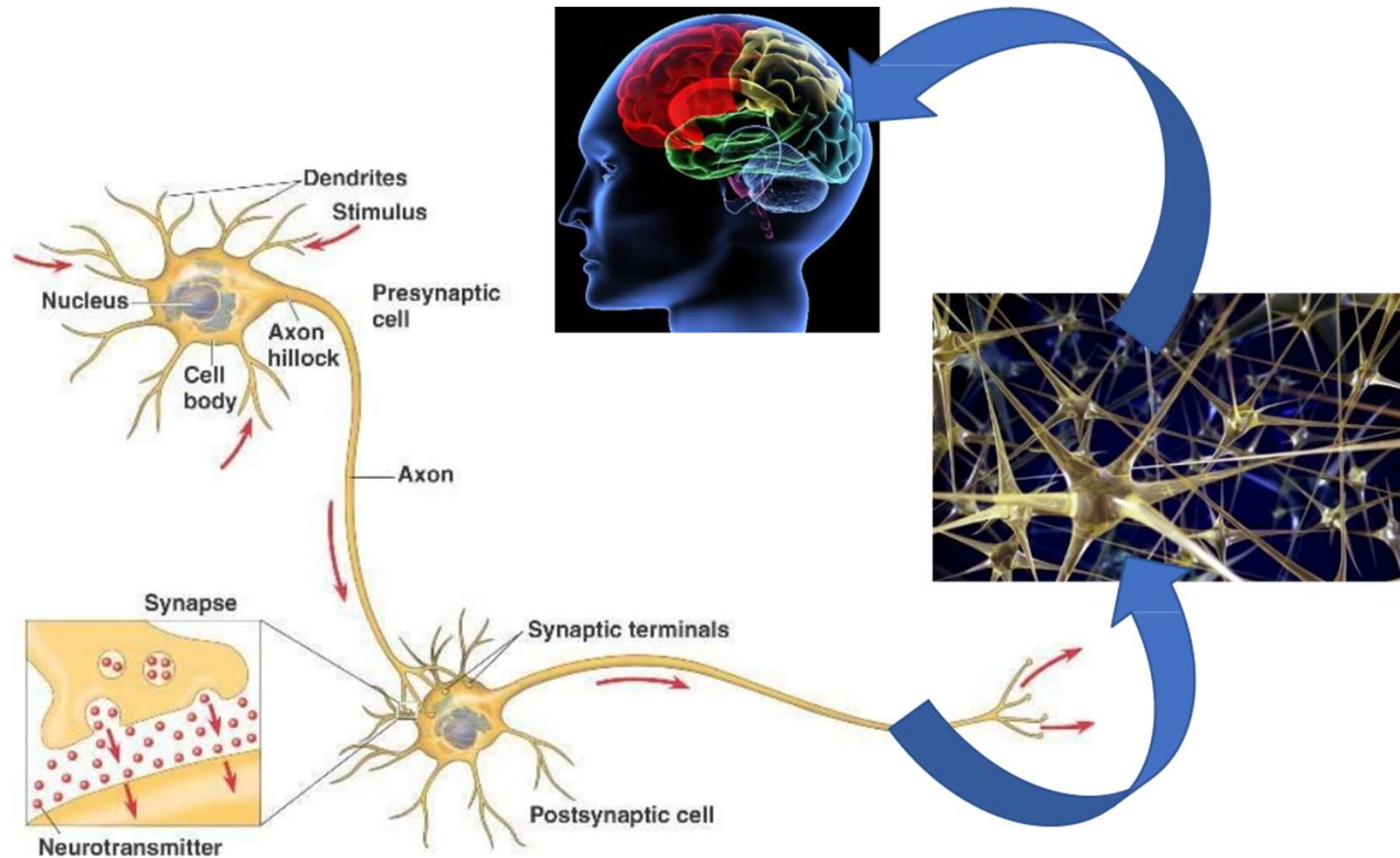


- **深度学习**

- 只需要改变step 1 !



• 人类的大脑



• 监督学习

- 监督学习 (supervised learning) 擅长在“给定输入特征”的情况下预测标签。每个“**特征-标签**”对都称为一个样本 (example)。有时，即使标签是未知的，样本也可以指代输入特征。我们的目标是学习一个模型，能够将任何输入特征映射到标签（即预测）。
- 监督学习的学习过程一般可以分为三大步骤：
 - 从**已知大量数据样本**中随机选取一个子集，为每个样本获取真实标签。有时，这些样本已有标签（例如，手写数字图片中的数字值）；有时，这些样本可能需要被人工标记（例如，图像分类）。这些输入和相应的标签一起构成了训练数据集；
 - 选择有监督的学习算法，它将训练数据集作为输入，并输出一个“已完成学习的模型”；
 - 将**之前没有见过的样本**特征放到这个“已完成学习的模型”中，使用模型的输出作为相应标签的预测。

- 监督学习—分类问题

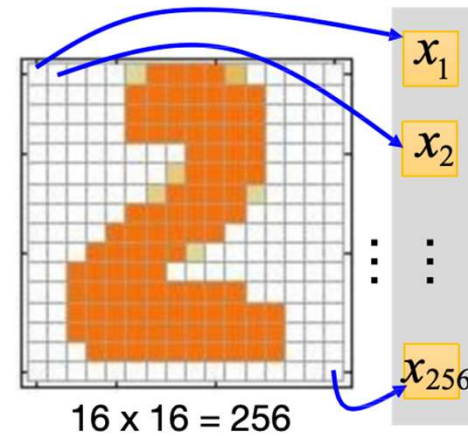
- 定义：在机器学习中，分类问题是指一种任务，其中模型需要学习如何将输入数据（通常是一组特征或属性）映射到离散标签或类别上。这种类型的问题是监督学习的一个主要分支，其中数据集包含已知类别标签的示例，模型通过这些示例学习如何预测新数据的类别。



- 监督学习—分类问题（例子）

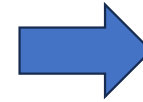
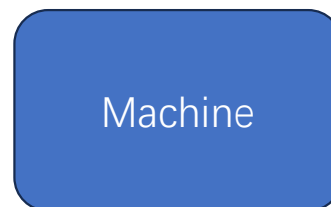
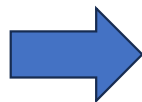
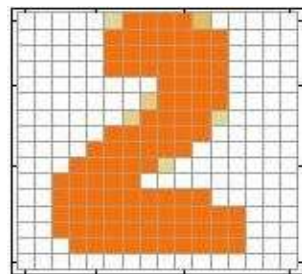
- 手写数字识别

- 输入：图片向量化



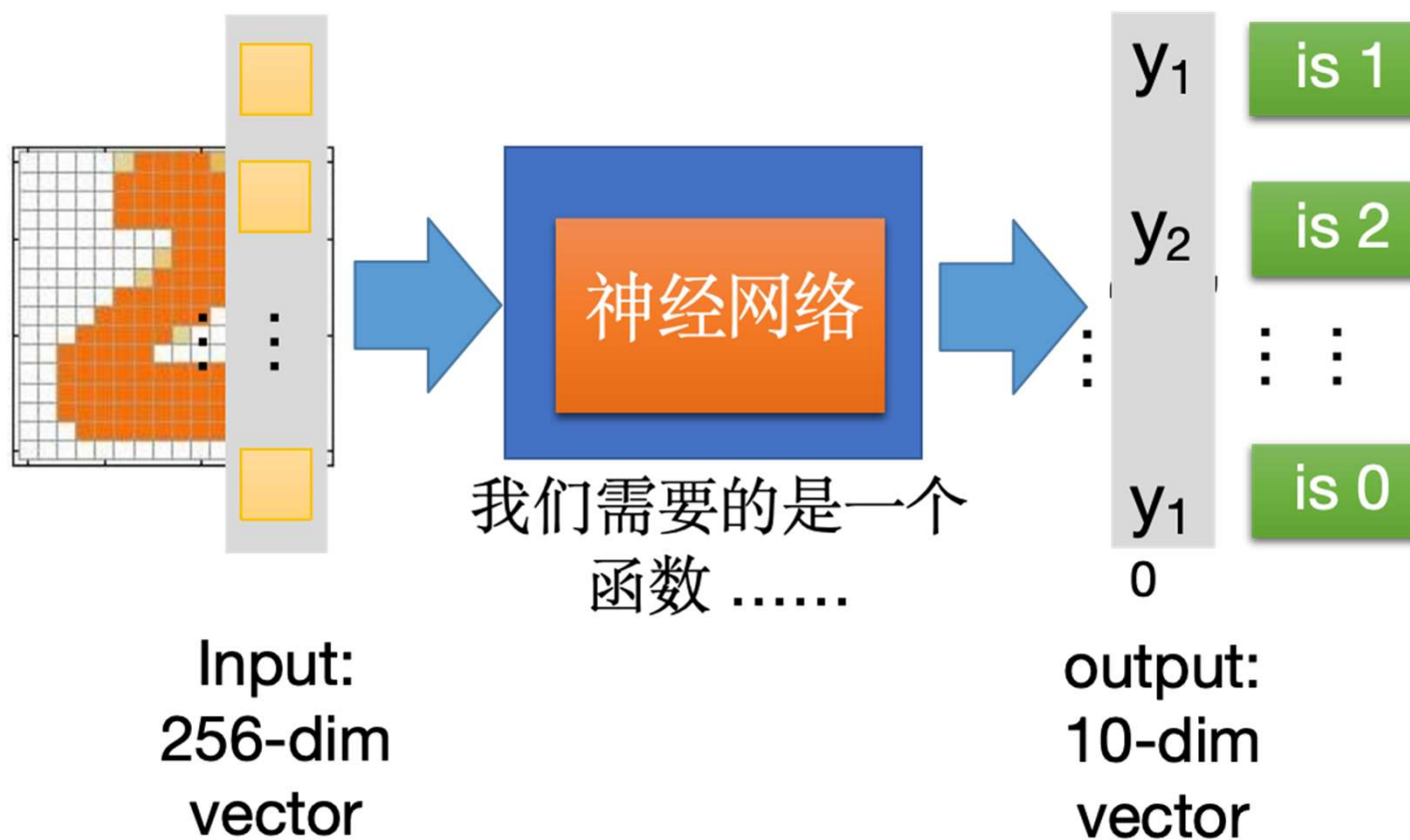
- 输出：图片属于不同类别的概率

Ink $\rightarrow 1$
No ink $\rightarrow 0$

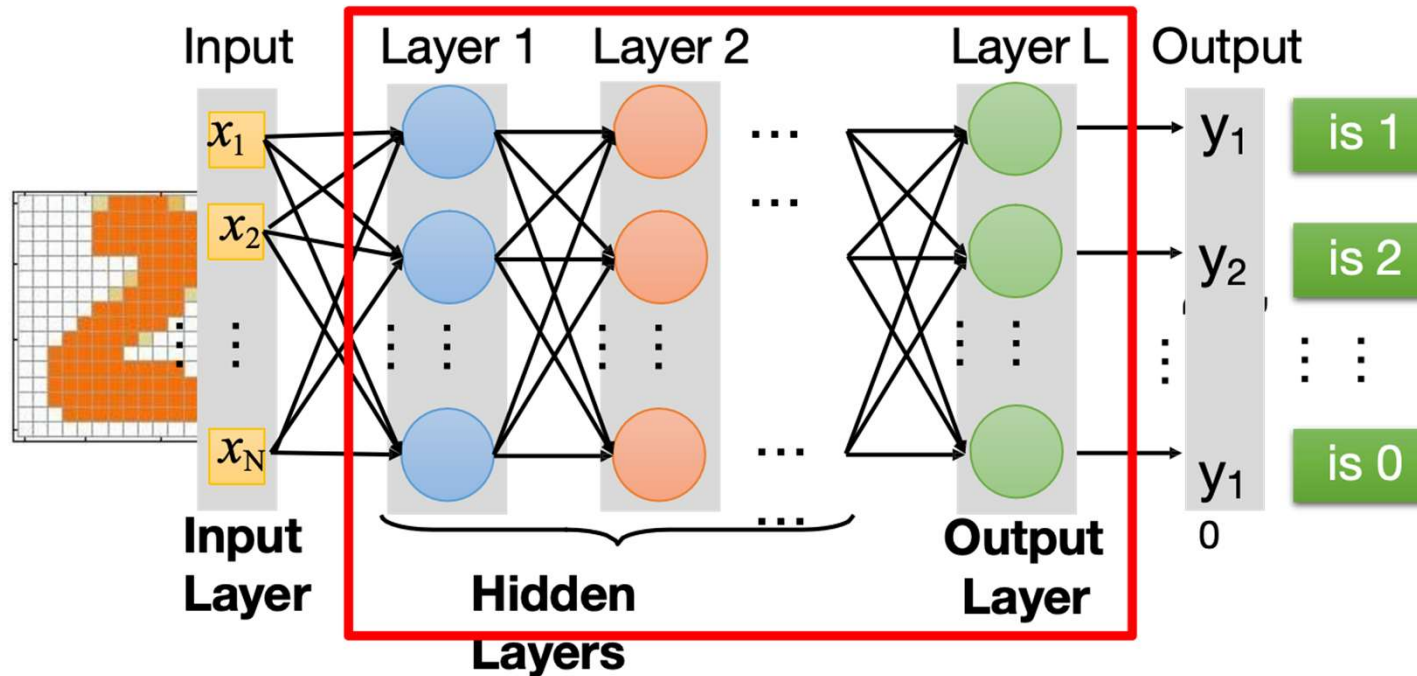


y_1	0.05
y_2	0.88
$\vdots$	
y_{10}	0.02

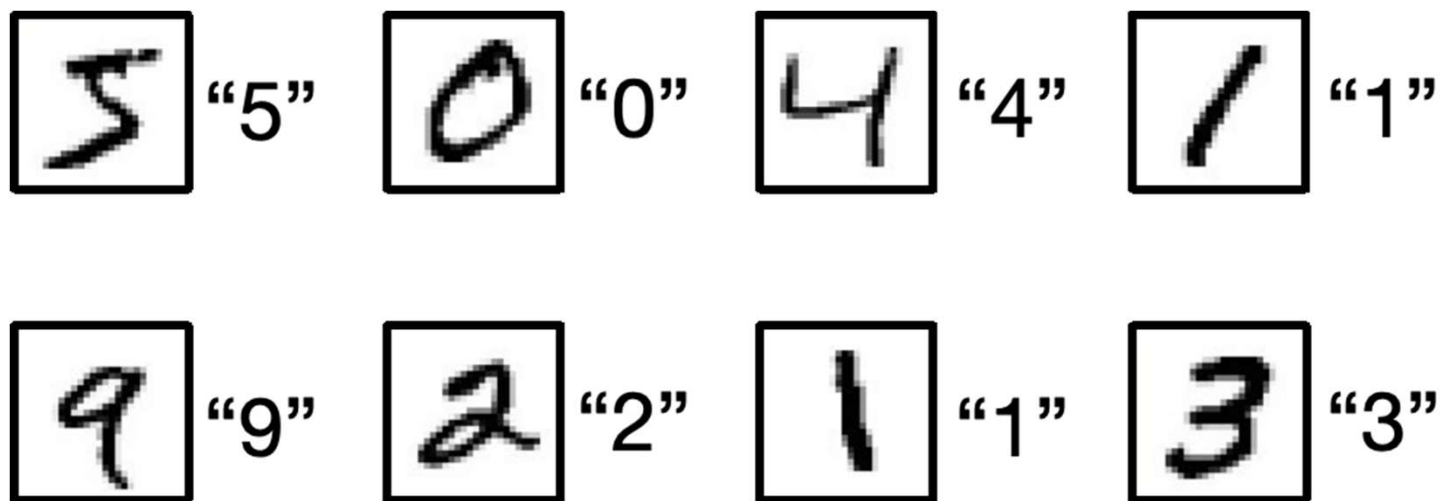
- 监督学习—分类问题（例子）
 - 手写数字识别
 - 模型结构



- 监督学习—分类问题（例子）
 - 手写数字识别
 - 模型结构（细节）

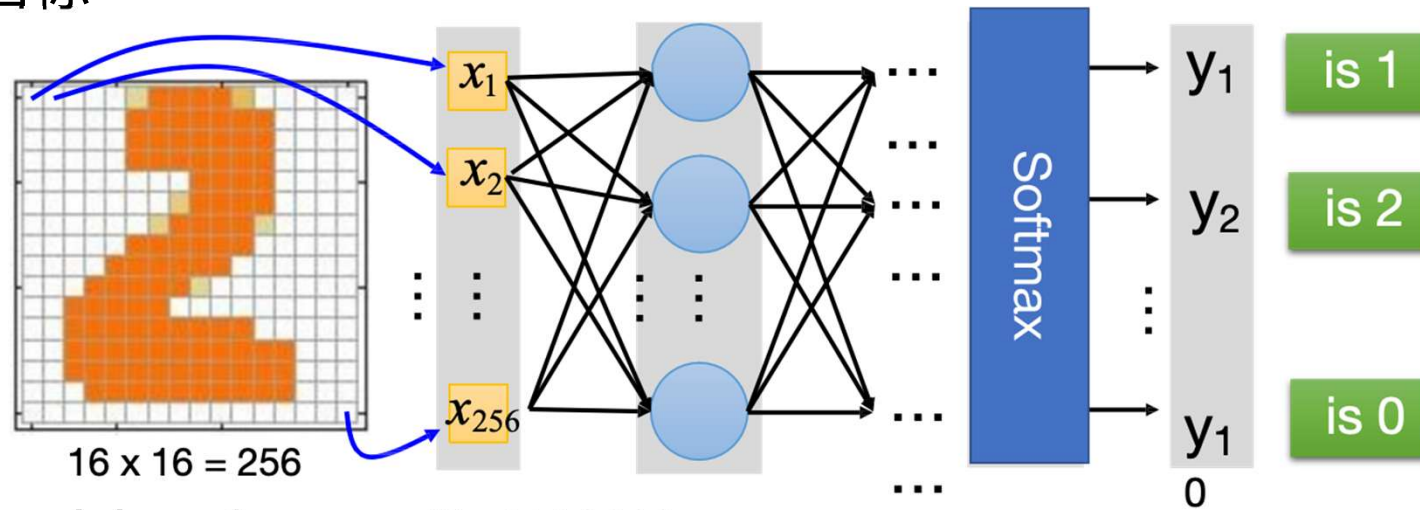


- 监督学习—分类问题（例子）
 - 训练数据



学习目标是在训练数据上定义的

- 监督学习—分类问题（例子）
 - 学习目标



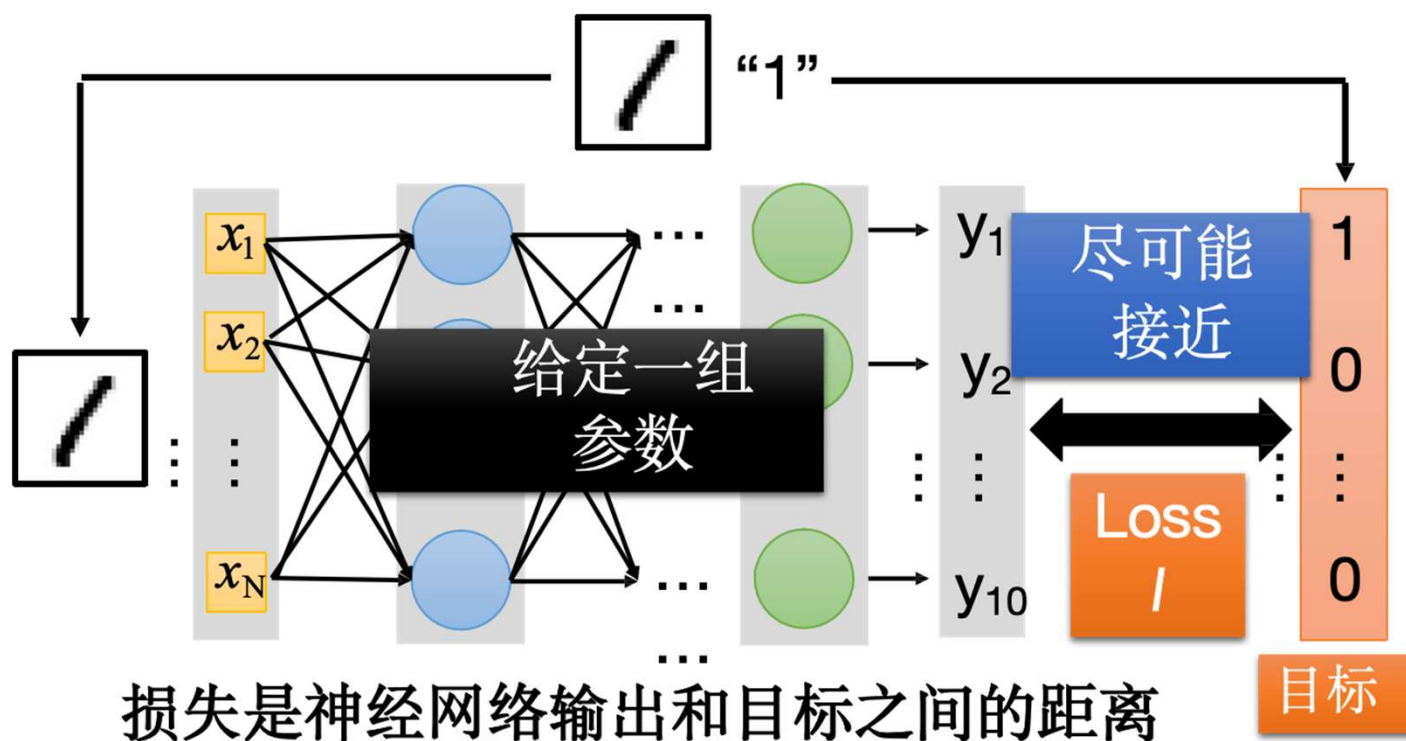
Ink $\rightarrow$ 1
No ink $\rightarrow$ 0

学习目标是

Input:  $\rightarrow$ y_1 是最大输出

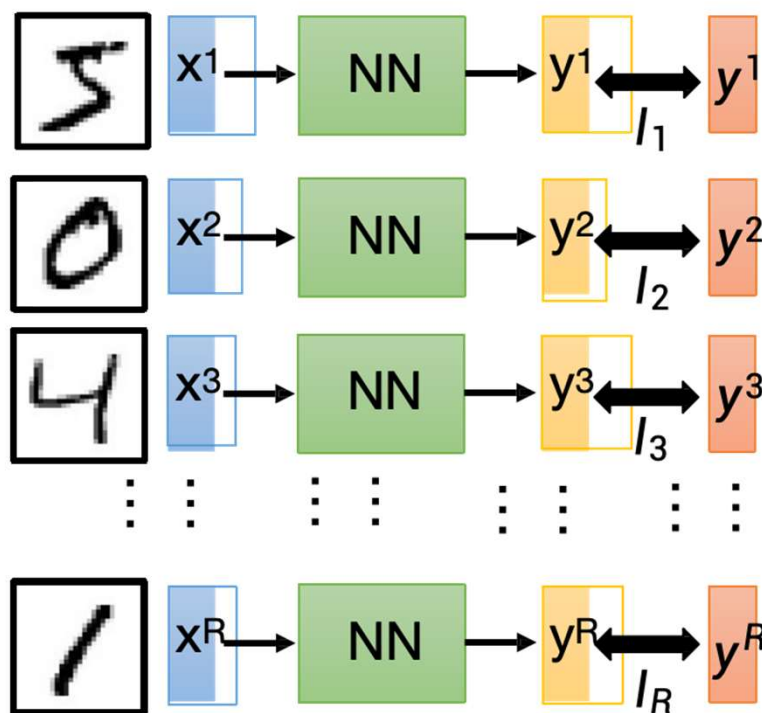
Input:  $\rightarrow$ y_2 是最大输出

- 监督学习—分类问题（例子）
 - 损失



- 监督学习—分类问题（例子）
 - 损失

对所有的训练数据 ...



损失之和尽可能的小

找到一个函数使损失函数 L 最小

找到参数 θ 使损失函数 L 最小

• 监督学习—回归问题

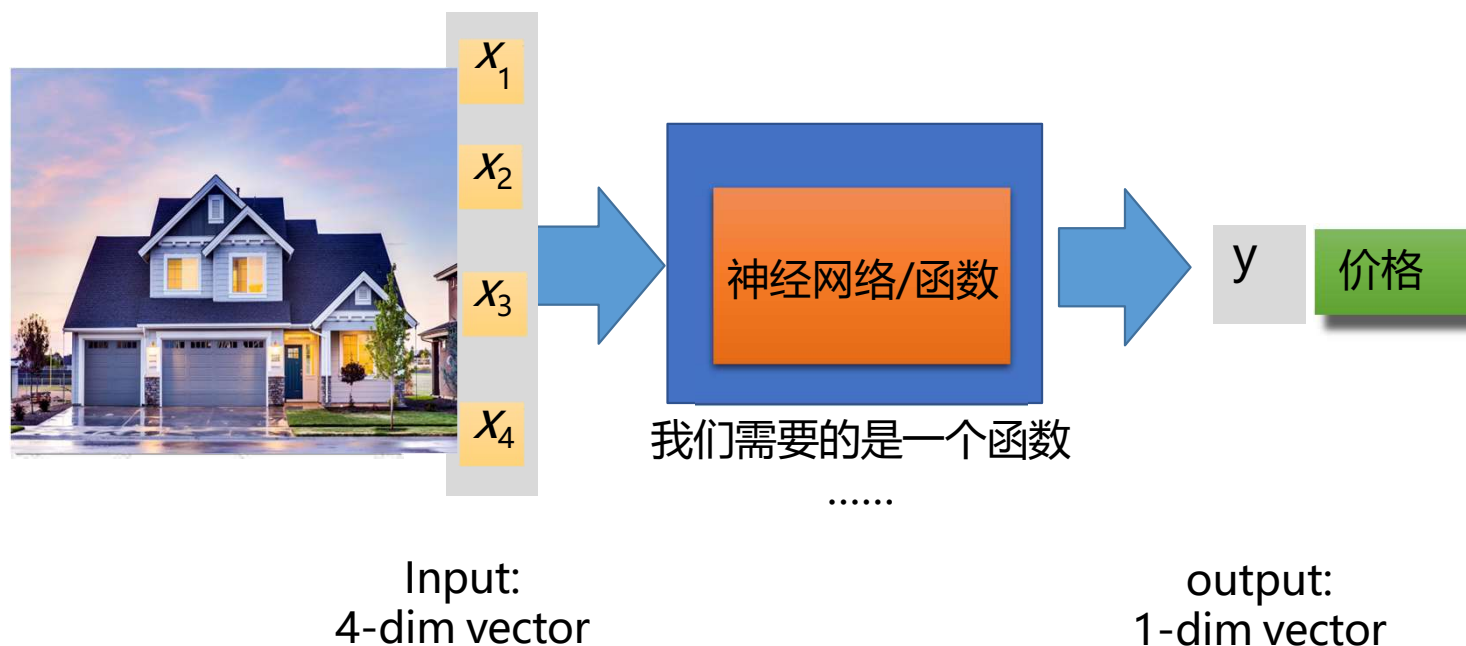
- 定义：回归问题是最简单的监督学习任务之一，模型通常学习如何讲输入的数据（通常是一组特征和属性）映射到一个连续的值。例如：预测房屋价格，预测股票价格，预测汽车的燃油效率等。

	房屋面积	卧室数量	浴室数量	步行距离	价格
	150	2	1	10	200,000
	200	3	2	15	350,000
	220	2	2	30	400,000

根据房屋面积、卧室数量、浴室数量和步行距离预测房屋价格

• 监督学习—回归问题

- 定义：回归问题是最简单的监督学习任务之一，模型通常学习如何讲输入的数据（通常是一组特征和属性）映射到一个连续的值。例如：预测房屋价格，预测股票价格，预测汽车的燃油效率等。



- 输入： $\mathbf{x} = [x_1, x_2, \dots, x_m]^T$ ， m 为特征的数量，在房价预测中 $m = 4$
- 模型有 m 个权重系数和1个偏置量

$$\mathbf{w} = [w_1, w_2, \dots, w_m]^T, b$$

- 输出：

$$y = w_1 x_1 + w_2 x_2 + \dots + w_m x_m + b$$

- 向量化形式

$$y = \mathbf{w}^T \mathbf{x} + b$$

- 收集训练数据

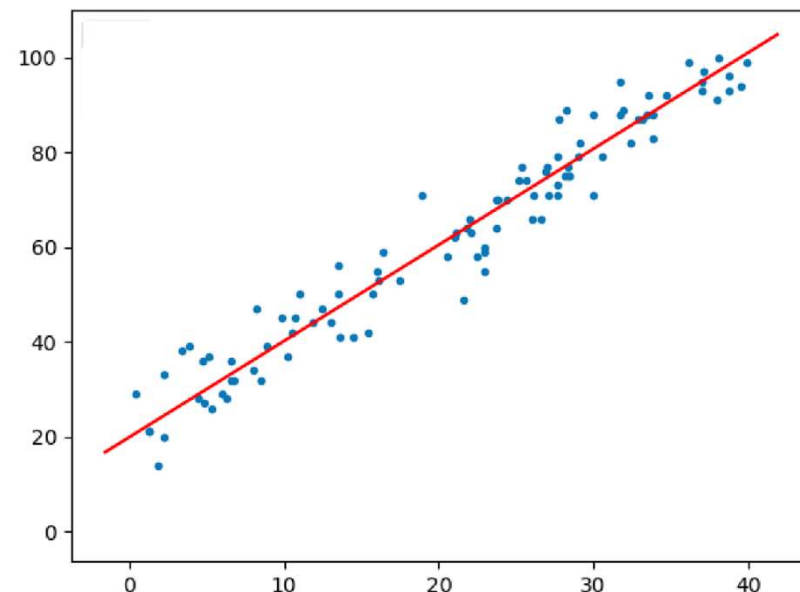
$$D = \{\mathbf{x}^{(i)}, y^{(i)}\}, i \in \{1, 2, \dots, N\}$$

- 单变量线性回归

- 如学习成绩与投入时间的关系，输入为标量

- 多变量线性回归

- 如前面提到的房价预测



	房屋面积	卧室数量	浴室数量	步行距离	价格
	150	2	1	10	200,000
	200	3	2	15	350,000
	220	2	2	30	400,000

- 定义目标函数，衡量拟合的效果
 - 最小二乘法：找到一条直线，使所有样本点到直线上预测点的均方误差 (mean square error, MSE) 最小

$$L(X, y, \mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} - b)^2$$

- 求解

$$\mathbf{w}^*, b^* = \operatorname{argmin}_{\mathbf{w}, b} L(X, y, \mathbf{w}, b)$$

- 解析解

$$L(\mathbf{X}, y, \mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} - b)^2$$

- 线性回归刚好是一个很简单的优化问题，我们可以通过求参数偏导，并令偏导为0，联立求解，最终也可以获得参数的解析解，这里将矩阵进行一定合并，写为如下形式：

$$\mathbf{Y} = (y^{(1)}, y^{(2)}, \dots, y^{(N)})^T, \mathbf{X} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \dots & \mathbf{x}^{(N)} \end{bmatrix}^T$$

- 获得解析解为：

$$(b, \mathbf{w}^T)^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

- 利用优化方法求解
 - 梯度下降算法调整参数，多次迭代

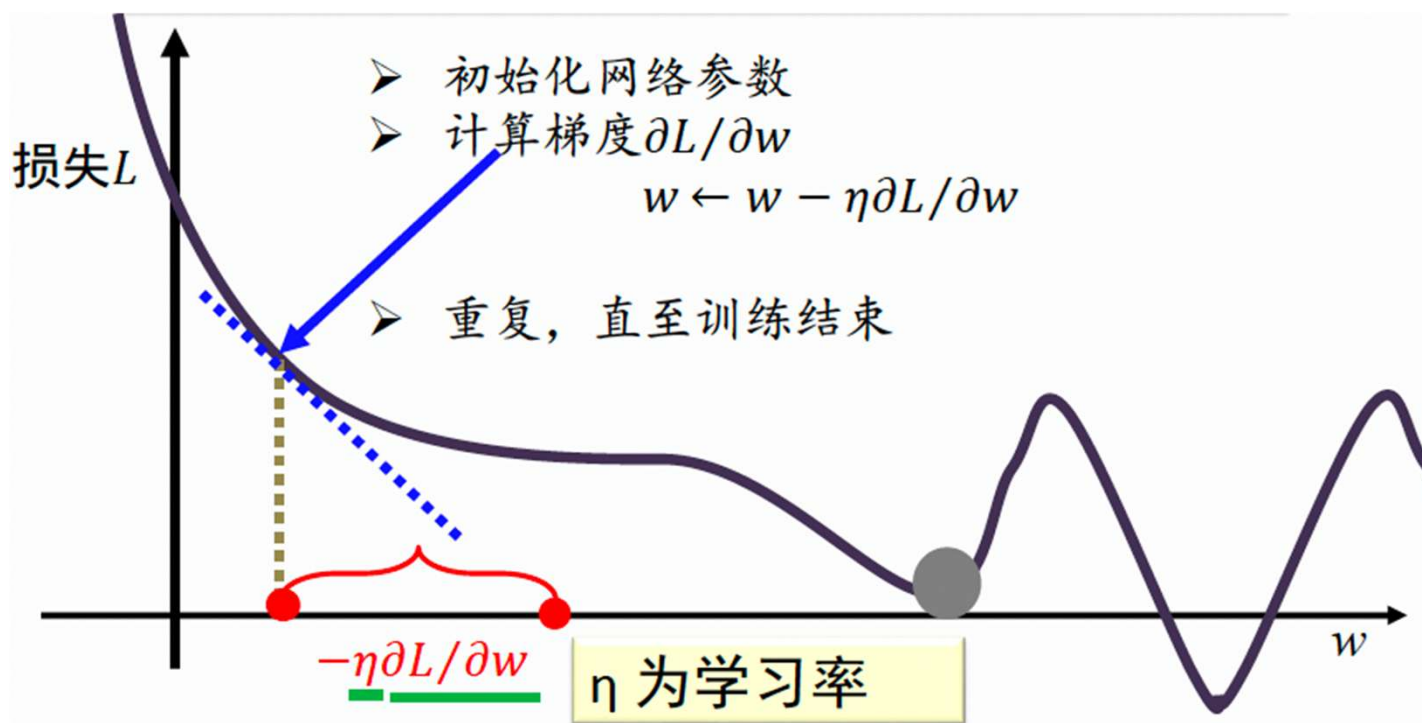
$$\mathbf{w} = \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}}$$

- 梯度：用于寻找更新权重的方向
- 学习率 η :
 - 一个在训练模型前设定的超参数
 - 用于指定更新参数的强度

- 梯度下降算法调整参数

$$w = w - \eta \frac{\partial L}{\partial w}, b = b - \eta \frac{\partial L}{\partial b}$$

- 学习一组参数(b, w)使得损失函数最小化



- 总结
 - 第一步：初始化模型参数的值，如随机初始化
 - 第二步：从数据集中使用所有样本计算损失函数关于模型参数的梯度，且在负梯度的方向上更新参数，并不断迭代这一步骤，直至模型有较好效果

given a starting point $x \in \text{dom } f$.

repeat

1. $\Delta x := -\nabla f(x)$.

2. *Line search*. Choose step size t via exact or backtracking line search.

3. *Update*. $x := x + t\Delta x$.

until stopping criterion is satisfied.

- 正态分布和线性回归之间的关系很密切。正态分布 (normal distribution)，也称为高斯分布 (Gaussian distribution)，最早由德国数学家高斯 (Gauss) 应用于天文学研究。简单的说，若随机变量 x 具有均值 μ 和方差 σ^2 (标准差 σ)，其正态分布概率密度函数如下：

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} (x - \mu)^2\right)$$

- 均方误差损失函数（简称均方损失）可以用于线性回归的一个原因是：我们假设了观测中包含噪声，其中噪声服从正态分布。噪声正态分布如下式：

$$y = \mathbf{w}^T \mathbf{x} + b + \epsilon$$

- 其中 $\epsilon \sim N(0, \sigma^2)$
- 因此，我们现在可以写出通过给定的 x 观测到特定 y 的似然 (likelihood) :

$$P(y|\mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} (y - \mathbf{w}^T \mathbf{x} - b)^2\right)$$

- 根据极大似然估计法，参数 W 和 b 的最优值是使整个数据集的似然最大的值：

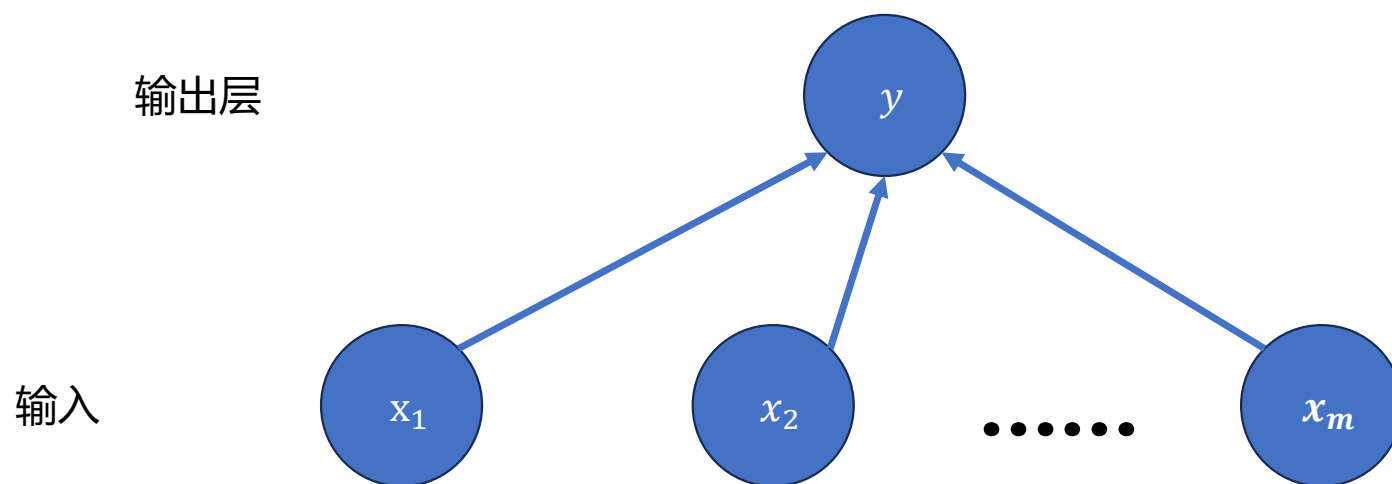
$$P(y|\mathbf{x}) = \prod_{i=1}^N p(y^{(i)}|\mathbf{x}^{(i)})$$

- 使用极大似然估计法，最大化 $P(y|\mathbf{x})$ ，相当于最小化负对数似然 $-\log P(y|\mathbf{x})$ 由此可以得到的数学公式是：

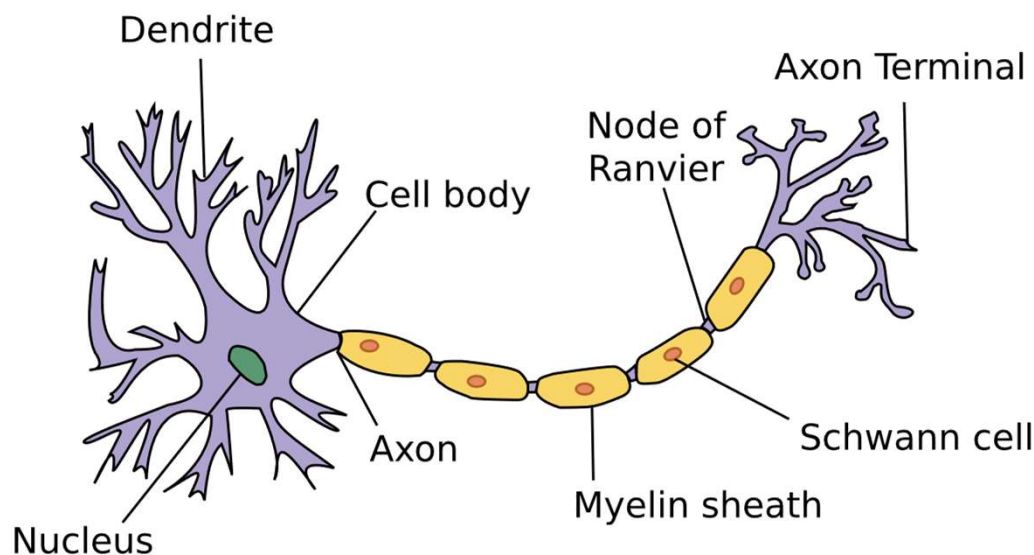
$$-\log P(y|\mathbf{x}) = \sum_{i=1}^N \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} - b)^2$$

- 假设 σ 是一个固定的常数，即可忽略第一项，第二项除了系数，是与均方误差一致的，因此，在高斯噪声的假设下，最小化均方误差等价于对线性模型的极大似然估计。

- 线性回归方法是最简单的单层单节点的网络



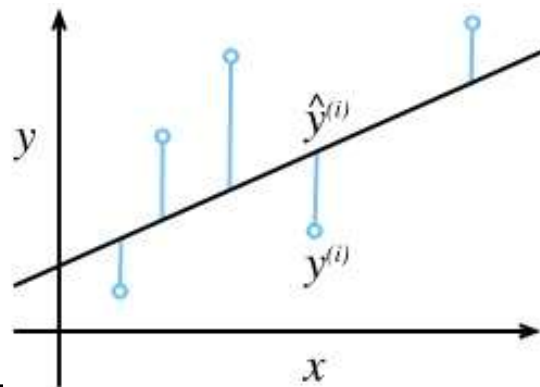
- 线性回归发明的时间（1795年）早于计算神经科学，但当控制学家、神经生物学家沃伦·麦库洛奇和沃尔特·皮茨开始开发人工神经元模型时，将线性回归作为了起点



由树突（dendrites, 输入终端）、细胞核（nucleus, CPU）组成的生物神经元图片。轴突（axon, 输出线）和轴突端子（axon terminal, 输出端子）通过突触（synapse）与其他神经元连接

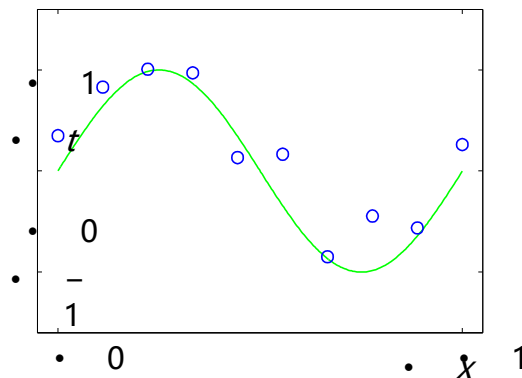
- 树突中接收到来自其他神经元（或视网膜等环境传感器）的信息 x_i 。该信息通过突触权重 w_i 来加权，以确定输入的影响（即，通过 $x_i w_i$ 相乘来激活或抑制）。
- 来自多个源的加权输入以加权和 $y = \sum_i x_i w_i + b$ 的形式汇聚在细胞核中，然后将这些信息发送到轴突 y 中进一步处理，通常会通过 $\sigma(y)$ 进行一些非线性处理。
- 之后，它要么到达目的地（例如肌肉），要么通过树突进入另一个神经元。

- 简单的线性回归



$$\hat{y} = wx + b$$

- 多项式回归



可以继续用 $\hat{y} = wx + b$ 么?

显然点的分布更符合高维多项式的形式, 即:

$$\hat{y} = w_3x^3 + w_2x^2 + w_1x + w_0$$

- 简单的线性回归

$$\hat{y} = wx + b$$

- 简单线性回归的Loss为：

$$L(X, y, w, b) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - wx^{(i)} - b)^2$$

- 令： $Y = (y^{(1)}, y^{(2)}, \dots, y^{(N)})^T$, $X = \begin{bmatrix} 1 & 1 & \dots & 1 \\ x^{(1)} & x^{(2)} & \dots & x^{(N)} \end{bmatrix}^T$
- 可以获得解析解：

$$(b, w)^T = (X^T X)^{-1} X^T Y$$

- 多项式回归

$$\hat{y} = w_3 x^3 + w_2 x^2 + w_1 x + w_0$$

- 参考简单线性回归中将输入数据 $x^{(i)}$ 矩阵化的思想, 也可以将多项式回归中的数据进行矩阵化, 此时有:

$$X = \begin{bmatrix} 1 & x^{(1)} & (x^{(1)})^2 & (x^{(1)})^3 \\ 1 & x^{(2)} & (x^{(2)})^2 & (x^{(2)})^3 \\ \dots & \dots & \dots & \dots \\ 1 & x^{(N)} & (x^{(N)})^2 & (x^{(N)})^3 \end{bmatrix}$$

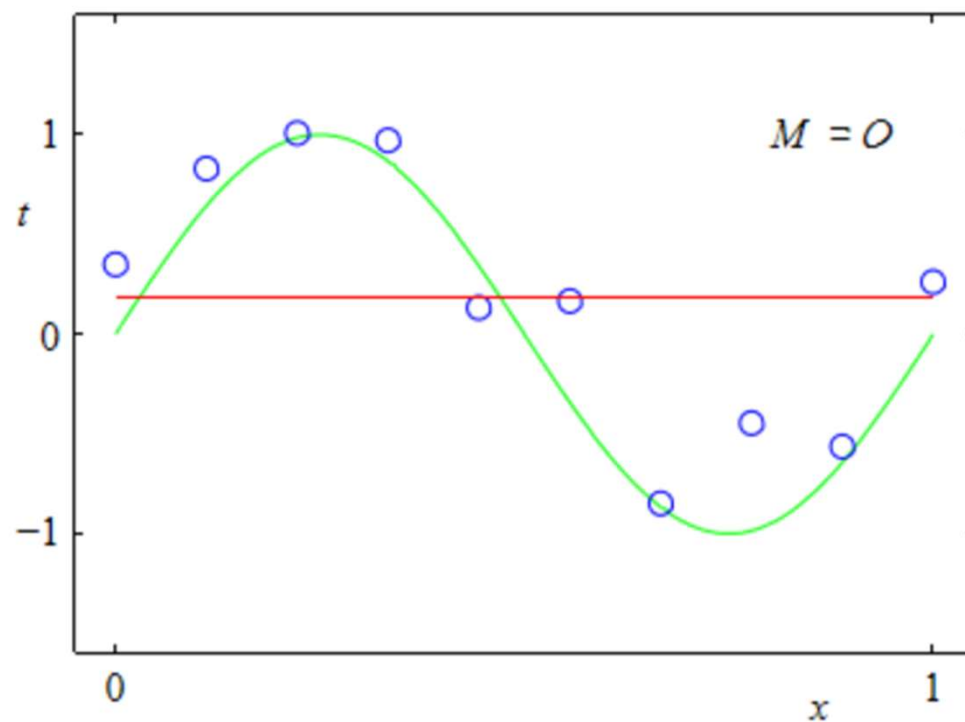
- 在MSE损失下, 最优的解析解为 $(w_0, w_1, w_2, w_3)^T = (X^T X)^{-1} X^T Y$, 形式与简单线性回归一致。

- MSE损失越低，就代表模型越好么？
 - 损失函数通常是模型效果的一种间接的表示，对训练一般来讲，在**训练数据**上的损失越低，代表模型数据拟合的好，但不代表在**测试数据**上有较好的效果。
 - **训练数据**：指已知的数据，在监督学习中，训练数据都是已知标签的
 - **测试数据**：使用模型未曾见过的数据，通常从真实世界中收集，会保证与训练的数据有一定的相似性。
 - **验证数据**：已知数据，但未用于训练，来评估模型的性能

- 训练误差：模型在训练数据集上得到的误差
- 泛化误差：模型从原始样本分布中抽取无限多未见过的数据样本得到的误差的期望
- 泛化误差永远不能准确地计算出。这是因为无限多的数据样本是一个虚构的对象。在实际中，只能通过将模型应用于一个独立的测试集来估计泛化误差，该测试集由随机选取的、未曾在训练集中出现的数据样本构成。

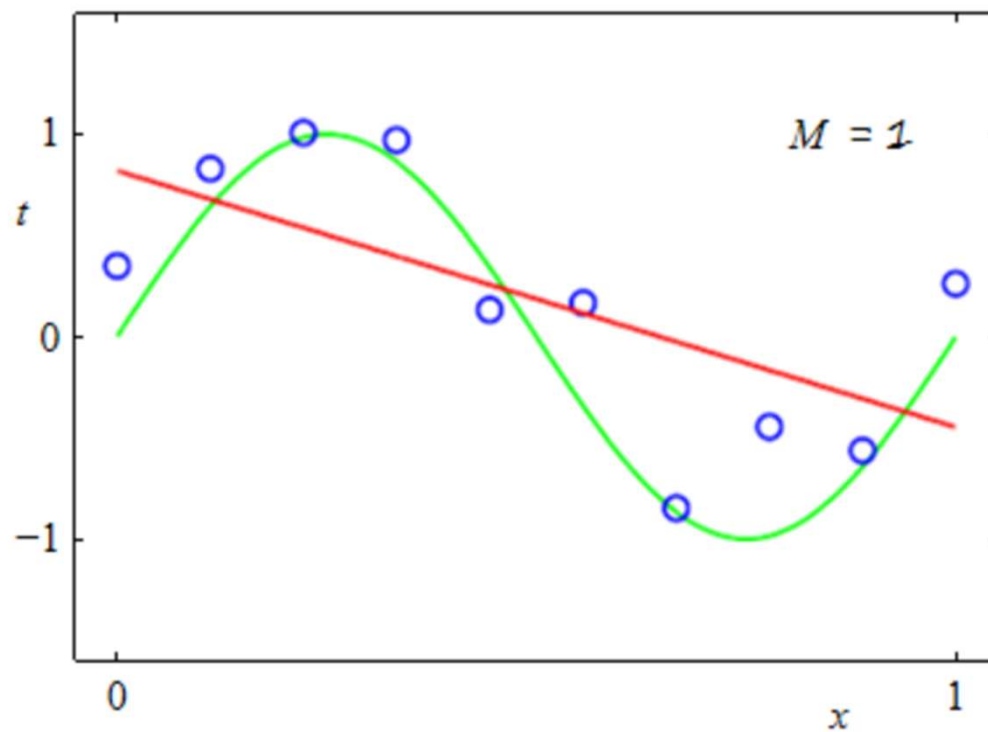
- 多项式阶数对拟合效果的影响：0阶

$$\hat{y} = w_0$$



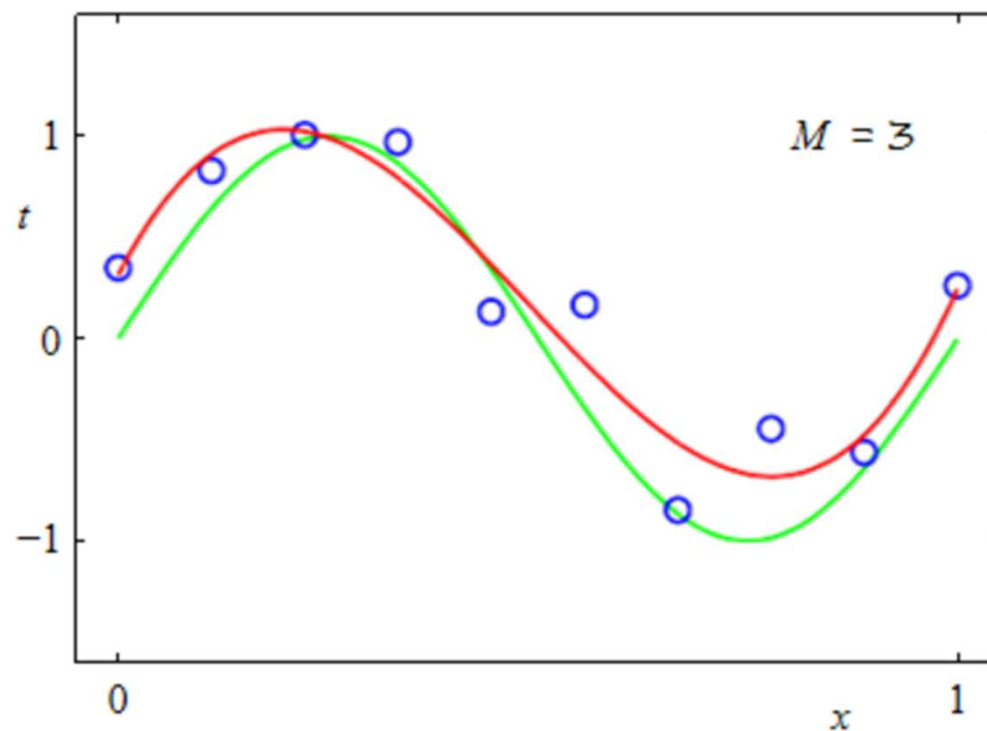
- 多项式阶数对拟合效果的影响：1阶

$$\hat{y} = w_1 x + w_0$$



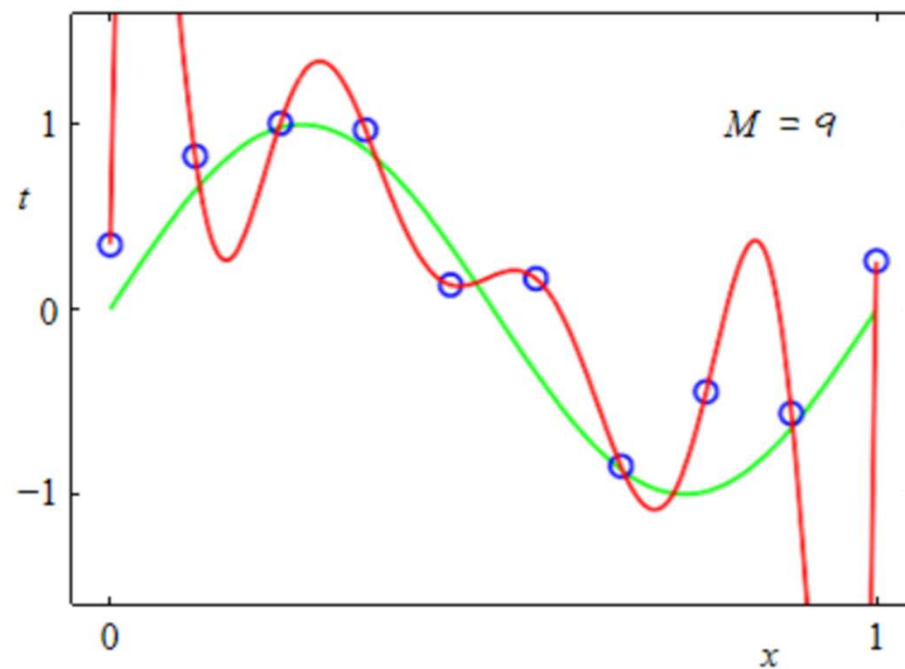
- 多项式阶数对拟合效果的影响：3阶

$$\hat{y} = w_3x^3 + w_2x^2 + w_1x + w_0$$

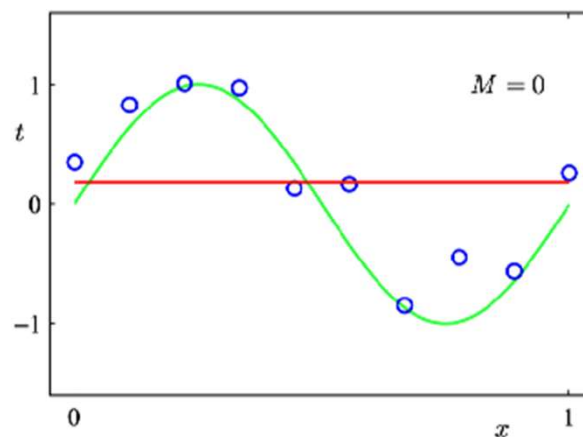


- 多项式阶数对拟合效果的影响：9阶

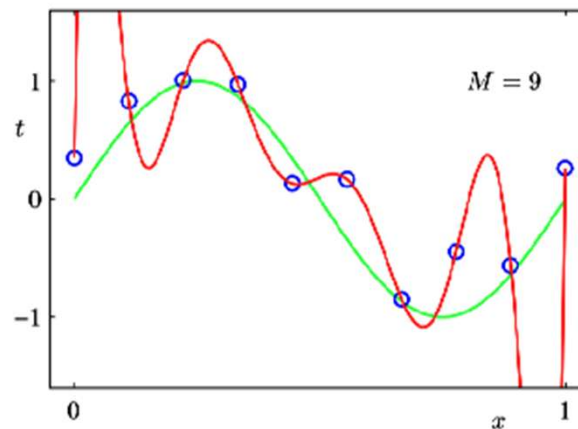
$$\hat{y} = w_9 x^9 + \cdots + w_1 x + w_0$$



- 欠拟合：模型太简单-不能拟合数据

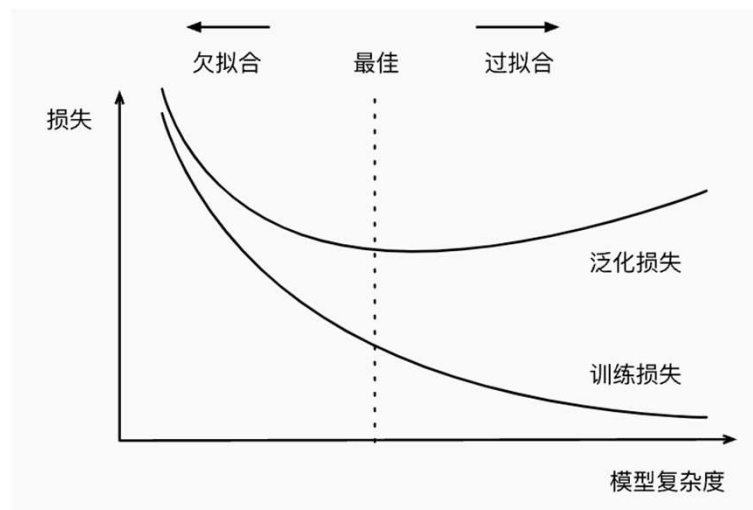


- 过拟合：模型过于复杂-拟合过于完美，不具备泛化性



• 模型复杂度

- 我们的特征是 x 的幂给出的，模型的权重是 w_i 给出的，偏置是 w_0 给出的。多项式回归可以看成简单线性回归的变种，仍然可以使用平方误差作为我们的损失函数。
- 在固定训练数据集的情况下，高阶多项式函数相对于低阶多项式的训练误差应该始终更低（最坏也是相等）。理论上，函数阶数等于数据样本数量的多项式函数可以完美拟合训练集。



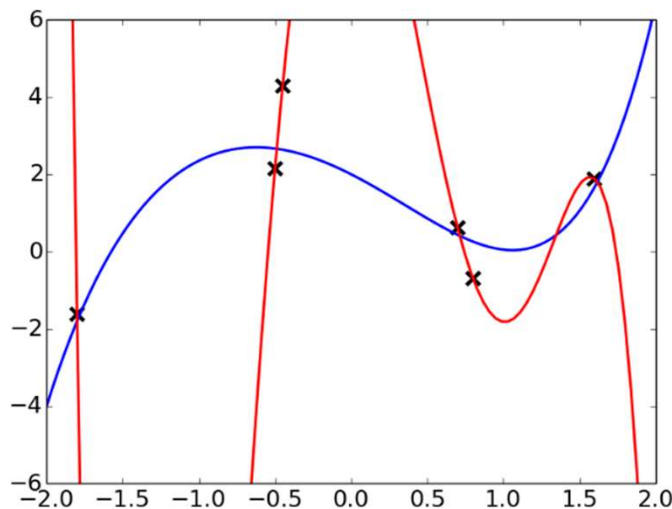
- 多项式的维度是一个超参数，我们可以使用验证集对其进行调整
- 但是限制维度是一个粗略的解决方法，因为即使数据量足够大，你没有能力学习到更复杂的模型
- 另一种方法：保持很大的模型，但是要对其进行正则化
 - 正则化：用于量化我们对一个假设相对于另一个假设的偏好程度的函数
 - L_1 或 L_2 正则化

• 向量范数

○ 假设 X 是 m 维向量，常见范数定义为：

- L_1 范数（曼哈顿范数）： L_1 范数是向量中各个元素的绝对值之和。对于 m 维向量 X ，其 L_1 范数记作 $\|X\|_1$ ，计算方式如下： $\|X\|_1 = |x_1| + |x_2| + \dots + |x_n|$
- L_2 范数（欧几里得范数）： L_2 范数是向量中各个元素的平方和的平方根。对于 m 维向量 X ，其 L_2 范数记作 $\|X\|_2$ ，计算方式如下： $\|X\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$
- 无穷范数（ L_∞ 范数）：无穷范数是向量中各个元素的绝对值的最大值。对于 m 维向量 X ，其无穷范数记作 $\|X\|_\infty$ ，计算方式如下： $\|X\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|)$

- 观察：过拟合的多项式通常有很大的系数



$$y = 0.1x^5 + 0.2x^4 + 0.75x^3 - x^2 - 2x + 2$$

$$y = -7.2x^5 + 10.4x^4 + 24.5x^3 - 37.9x^2 - 3.6x + 12$$

- 所以让我们试着将系数的值保持很小

- 我们希望权重的值变小的另一个原因是：
- 假设所有的训练例子中的输入 x^1 和 x^2 几乎都是相等的。以下两种权重取值会做出几乎相同的预测：

$$\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \mathbf{w} = \begin{pmatrix} -9 \\ 11 \end{pmatrix}$$

- 但是，如果测试分布稍微不同，第二种权重取值可能会做出错误的预测（比如： x^1 和 x^2 相差得较大）。

- 我们可以通过选择加上 L_2 惩罚项作为我们的正则化器，来鼓励权重变得较小

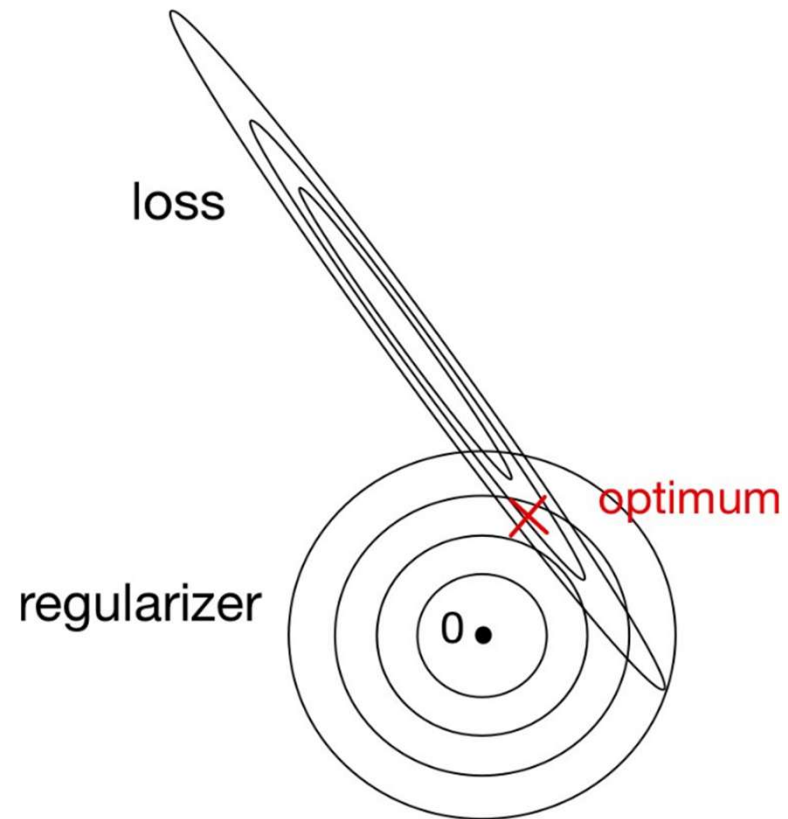
$$R(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \sum_j w_j^2$$

- 注意：要严格来说， L_2 范数是欧几里德距离，所以我们实际上是在正则化 L_2 范数的平方
- 正则化后的损失函数在拟合数据和权重的范数之间进行权衡

$$L_{reg} = L + \lambda R = L + \frac{1}{2} \lambda \sum_j w_j^2$$

- 在这里， λ 是一个可以使用验证集来调整的超参数

• 可视化



- 回想一下梯度下降更新:

$$\mathbf{w} = \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

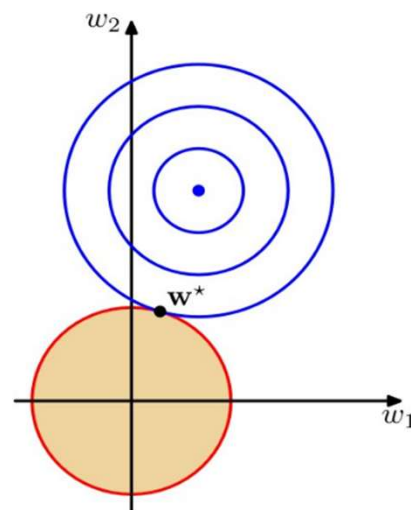
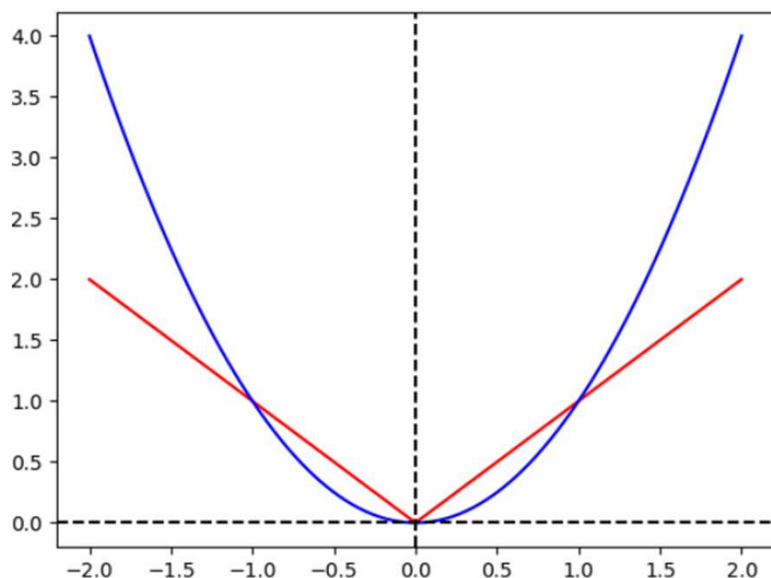
- 正则化损失的梯度下降更新也可称为权重衰减(weight decay):

$$\begin{aligned}\mathbf{w} &= \mathbf{w} - \alpha \left(\frac{\partial L}{\partial \mathbf{w}} + \lambda \frac{\partial R}{\partial \mathbf{w}} \right) \\ &= \mathbf{w} - \alpha \left(\frac{\partial L}{\partial \mathbf{w}} + \lambda \mathbf{w} \right) \\ &= (1 - \alpha \lambda) \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}\end{aligned}$$

- L_2 范数对比 L_1 范数

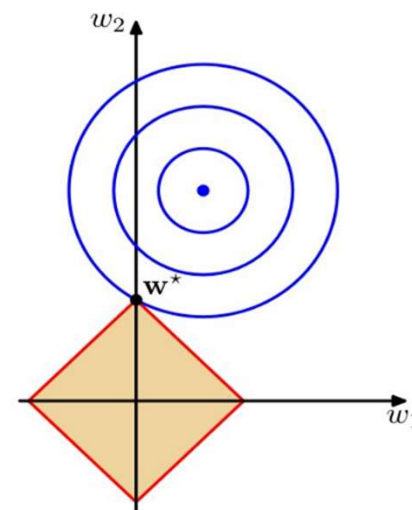
- L_2 正则化线性模型构成经典的岭回归 (ridge regression) 算法, L_1 正则化线性回归是统计学中类似的基本模型, 通常被称为套索回归 (lasso regression) 。
- 使用 L_2 范数的一个原因是它对较大的权重向量惩罚较大。这使得我们的学习算法偏向于在大量特征上均匀分布权重的模型。在实践中, 这可能使它们对单个变量中的观测误差更为稳定。
- 相比之下, L_1 惩罚会导致模型将权重集中在一小部分特征上, 而将其他权重清除为零。这称为特征选择 (feature selection), 这可能是其他场景下需要的。

- L1范数（绝对值之和）是另一种正则化器，它鼓励权重值恰好为零
- 我们可以根据我们希望得到的特性来设计正则化器



L2 regularization

$$\mathcal{R} = \sum_i w_i^2$$



L1 regularization

$$\mathcal{R} = \sum_i |w_i|$$

THANK YOU!

Jifeng Dai
Department of Electronic Engineering
daijifeng@tsinghua.edu.cn

